CABIOS

Vol.9. no 3. 1993
Pages 267 – 273

Multiple sequence alignment by parallel simulated annealing

Masato Ishikawa, Tomoyuki Toya, Masaki Hoshida, Katsumi Nitta, Atsushi Ogiwara[1] and Minoru Kanehisa[1]

Abstract

We have developed simulated annealing algorithms to solve the problem of multiple sequence alignment. The algorithm was shown to give the optimal solution as confirmed by the rigorous dynamic programming algorithm for three-sequence alignment. To overcome long execution times for simulated annealing, we utilized a parallel computer. A sequential algorithm, a simple parallel algorithm and the temperature parallel algorithm were tested on a problem. The results were compared with the result obtained by a conventional tree-based algorithm where alignments were merged by two-way dynamic programming. Every annealing algorithm produced a better energy value than the conventional algorithm. The best energy value, which probably represents the optimal solution, was reached within a reasonable time by both of the parallel annealing algorithms. We consider the temperature parallel algorithm of simulated annealing to be the most suitable for finding the optimal multiple sequence alignment because the algorithm does not require any scheduling for optimization. The algorithm is also useful for refining multiple alignments obtained by other heuristic methods.

Introduction

The alignment is a basic tool for obtaining biological insights from the sequence data of proteins and nucleic acids. Pairwise alignments have been widely used in database searching in order to extend sequence similarity to functional similarity. Here we consider the problem of multiple sequence alignment, or simultaneously aligning more than two entire sequences—a process of maximizing a measure of similarity over the length of the alignment . When a set of sequences are multiply aligned, it is possible to identify motifs and other functionally important sites along the sequence. Basically, there have been two approaches to solving the problem of multiple sequence alignment: rigorous optimization by dynamic programming and heuristic algorithms.

Theoretically, the dynamic programming algorithm for pairwise sequence alignment (Needleman and Wunsch, 1970; Smith and Waterman, 1981, Goad and Kanehisa, 1982) can easily be extended to multiple alignment of N sequences (Waterman *et al.*, 1976). However, this requires memory space for an N-dimensional array and calculation time of the order of the Nth power of the sequence length. Although a method was proposed to cut unnecessary computation (Carrillo and Lipman, 1988), it still needs too much computation to solve a practical alignment problem. In order to obtain approximate solutions within a practical time, a number of heuristic algorithms have been devised for multiple sequence alignment (Hogeweg and Hesper, 1984; Johnson and Doolittle, 1986; Bacon and Anderson, 1986; Feng and Doolittle, 1987; Taylor, 1987; Barton, 1990). Most of these are based on the combination of pairwise or two-way dynamic programming alignments.

In this paper we propose a simulated annealing algorithm to solve the problem of multiple sequence alignment. Simulated annealing is a stochastic algorithm to solve complex combinatorial optimization problems (Kirkpatrick *et al.*, 1983). In general, the solution obtained becomes closer and closer to the rigorous solution as more calculations are performed. When the algorithm is applied to multiple sequence alignment, it can evaluate all sequences simultaneously. Therefore, it is straightforward to incorporate complex terms e.g. position-dependent weights for conserved residues and various types of gaps, in the measure of similarity.

Since simulated annealing generally requires a long execution time, it is desirable to utilize the power of parallel computers where computation is performed in parallel in many processing elements. We have developed parallel versions of the simulated annealing algorithm and tested these on a Multi-PSI, a parallel machine developed at ICOT. Specifically, we applied the temperature parallel or time-homogeneous algorithm (Kimura and Taki, 1990), which eliminates the difficulty of finding an appropriate cooling schedule for annealing.

System and methods

Multi-PSI is an MIMD (multiple instruction multiple data) type parallel machine with 64 processing elements (PEs) (Uchida, 1992). Every PE has its own 80 Mbyte local memory, and no shared memory is furnished with this machine. The PEs are loosely connected by a two-dimensional 8 × 8 mesh. Generally speaking, a loosely connected parallel machine can have a larger configuration of processors than a shared memory parallel machine, but the former may suffer more from overheads of inter-processor communication. However, this was not a problem in our implementation of the parallel simulated

Institute for New Generation Computer Technology (ICOT), 1-4-28 Mita, Minato-ku, Tokyo 108 and [1]Institute for Chemical Research, Kyoto University, Uji, Kyoto 611, Japan

annealing algorithms, for the frequency of inter-processor communication was very low.

The simulated annealing algorithms and other conventional algorithms described in this paper are written in parallel logic programming language KL1 (Nakajima *et al.*, 1989). The programs are available from the authors upon request. KL1 is similar to Prolog in its syntax, but it can be used with different styles of programming. We adopted a process (object)-oriented programming style, where a program is composed of processes and message streams. One process receives a message from another process, does some work in response to it, and, usually, sends a new message to another process. As a result, the program is realized as a network of processes, and this logical network is mapped on a parallel computer.

Algorithm

Simulated annealing

An outline of the simulated annealing (SA) algorithm is as follows.

Let X be a solution space and $E{:}X \rightarrow \mathbf{R}$ be the objective function we want to minimize. Given an arbitrary initial solution $x_0 \in X$, the algorithm generates a sequence of solutions $\{x_n\}_{n = 0,1,2, \ldots N}$ iteratively as follows;

(i) Modify the current solution x_n randomly and get a candidate for the next solution x'_n.

(ii) Calculate the change in the objective function: $\Delta E = E(x'_n) - E(x_n)$.

(iii) When $\Delta E \leq 0$, accept the candidate: $x_{n+1} = x'_n$. When $\Delta E > 0$, accept the candidate with probability $p = \exp(-\Delta E/T_n)$ where $T_n > 0$ is a control parameters; and reject it otherwise: $x_{n+1} = x_n$.

For a large enough N, the algorithm outputs a final solution x_N.

When $T_n = +\infty$, SA is reduced to a blind random search; when $T_n = +0$, it is reduced to a greedy algorithm (iterative improvement) converging to one of the local optimals at hand. For $0 < T_n < +\infty$, it behaves in a manner between these two extremes. In the following, we refer to the operation (i) as a move and the sequence of operations (i)−(iii) as a step.

SA is based on an analogy between combinatorial optimization and statistical mechanics. Thus, the objective function E is referred to as the energy function and T_n as the temperature. SA has received much attention in combinatorial optimization problems since it can provide much better solutions than conventional heuristics, though it often requires lengthy execution time. At a constant temperature, $T_n = T$, SA simulates an equilibrium state of an imaginary physical system at temperature T. Hence, the solutions x_n of a combinatorial optimization problem is distributed according to the Boltzmann distribution at temperature T. This Boltzmann distribution converges to the lowest energy state (optimal solution) as the temperature decreases to zero. Thus, one might expect SA to provide the optimal solution in principle.

The sequence of decreasing values of $\{T_n\}_{n = 0,1,2 \ldots}$ determines a cooling schedule. It is well known that the cooling schedule has great influence on the performance of SA; a poor schedule only leads to poor solutions (local minima). It is here that the cooling schedule problem arises, i.e. how slowly the temperature should be decreased so as to obtain the best solution possible within a given number of annealing steps, i.e. within a given amount of computation time. Determining an ideal cooling schedule is often a difficult task, requiring much trial and error.

Parallelization

In order to accelerate the execution of SA, parallel algorithms have been studied extensively. We applied the following two parallel algorithms to the problem of multiple sequence alignment.

Simple parallel SA. A simple parallel algorithm is a naive combination of sequential SAs; every available PE has an initial solution and a cooling schedule using a distinct sequence of random numbers. All the final solutions obtained are compared with each other, and the best one—the one with the minimum energy value—is selected as the solution for the parallel algorithm. In each PE, a simple cooling schedule such as:

$$T_n = \alpha^{\lfloor n/K \rfloor} \cdot T_0 \quad n = 1,2, \ldots N;\ 0 < \alpha < 1;\ K >> 1$$

is often used (Kirkpatrick *et al.*, 1982). When the initial temperature T_0 and the final temperature T_N are chosen properly and both K and N/K (the number of so-called inner loops and outer loops) are large enough, the cooling schedule has been known to work well in many applications.

Temperature parallel SA. The basic idea of the algorithm is to use parallelism in temperature and to perform annealing processes concurrently at various temperatures (Kimura and Taki, 1990). One of the major advantages of the algorithm is that it automatically constructs an appropriate cooling schedule from a given set of temperatures. Hence, it partly solves the cooling schedule problem.

The outline of the algorithm is as follows. Each PE maintains one solution and performs the annealing process concurrently at a constant temperature that differs from PE to PE. After every k annealing steps, every pair of PEs with adjacent temperatures performs a probabilistic exchange of solutions. Let $p(T,E,T',E')$ denote the probability of the exchange between two solutions: one with energy E at temperature T and the other with energy E' at temperature T'. This is defined as follows:

$$p(T,E,T',E') = \begin{cases} 1 & \text{if } \Delta T \cdot \Delta E < 0 \\ \exp\left(-\dfrac{\Delta T \cdot \Delta E}{T \cdot T'}\right) & \text{otherwise} \end{cases}$$

where $\Delta T = T' - T$, $\Delta E = E' - E$

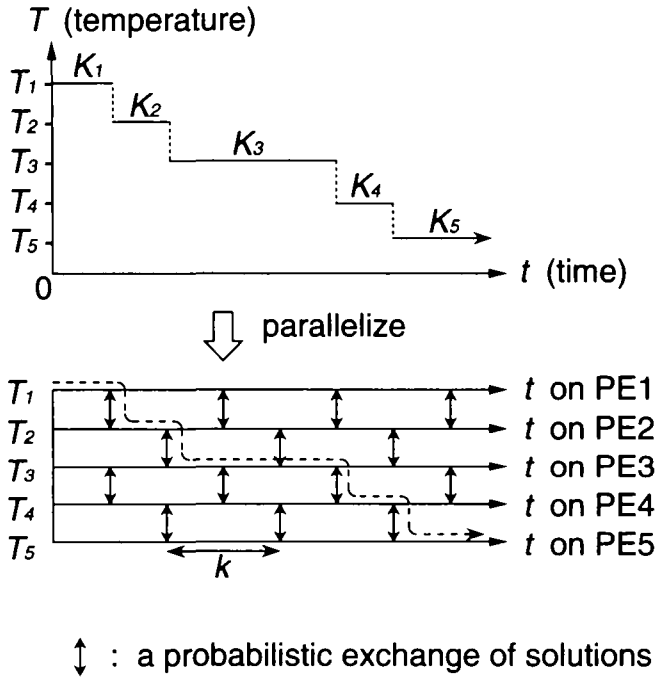

Fig. 1. Parallelization of sequential simulated annealing by temperature (Kimura and Taki, 1990). A cooling schedule for a sequential simulated annealing (**upper part**) is embedded in the parallel execution of processing elements, PEs (**lower part**). An appropriate cooling schedule is dynamically constructed from a given set of temperatures assigned to PEs, as indicated by the dotted line.

In order to avoid possible collisions between pairwise exchanges, the PEs are divided into two halves and each half is examined in turn at every $k/2$ steps, as illustrated in Figure 1. The algorithm can be stopped at any time after a large number of steps and we can find a well-optimized solution on the PE that has the lowest temperature.

Since the exchanging of solutions between processors with different temperatures is merely the changing of the temperature for each participant solution, each solution will select its appropriate cooling schedule dynamically through successive competitions with others for lower temperatures. The probability of exchange has been defined so that it is advantageous to the solutions with low energies. Hence, the solution at the lowest temperature, the winner of the competitions, is expected to be the best solution so far. The cooling schedule adopted by this winner, which is dynamically and stochastically determined, is supposed to be an efficient cooling schedule and is invisibly embedded in the parallel execution (Figure 1).

The temperature parallel algorithm is advantageous when we want to continue execution until a satisfactory solution is found. We can stop the execution at any time and examine whether a satisfactory solution has already been obtained. If not, we can resume the execution without any rescheduling. In contrast, in the simple parallel algorithm, when an obtained solution is not satisfactory, we have to reconsider the cooling schedule and repeat the time-consuming annealing process. It was reported

```
(a)          1         2         3         4         5         6
    12345678901234567890123456789012345678901234567890123456789012345678901234567890123456789
SKRV  :-------GFILATPQTGEASKNVISHVIHCLATIGKPHTIKTDNGPGYTGKNFQDFCQKLQI-------
MMTV  :-------YSHPTFATARTGEATKDVLQHLAQSFAYMGIPQKIKTDNAPAYVSRSIQEFLARW-------
IAP   :-------GVMFATTLTGEKASYVIQHCLEAWSAWGKPRIKTDNGPAYTSQKFRQFCRQMDVT-------
RSV   :-------IVVTQHGRVTSVAVQHHWATAIAVLGRPKAIKTDNGSCFTSKSTREWLARWGIAH-------
HTLV-1:-------SGAISATQKRKETSSEAISSLLQAIAHLGKPSYINTDNGRAYISQDFLNMCTSLA-------
HTLV-2:-------DTFSGAVSVSCKKKETSCETISAVLQAISLLGKPLHINTDNGPAFLSQEFQEFCT-------
BLV   :-------HASAKRGLTTQTTIEGLLEAIVHLGRPKKLNTDQGANYTSKTFVRFCQQFGVSLS-------
(energy = 877)

(b)          1         2         3         4         5         6
    12345678901234567890123456789012345678901234567890123456789012345678901234567890123456789
SKRV  :-------GFILATPQTGEASKNVISHVIHCLATIGKPHTIKTDNGPGYTGKNFQDFCQKLQI-------
MMTV  :-------YSHPTFATARTGEATKDVLQHLAQSFAYMGIPQKIKTDNAPAYVSRSIQEFLARW-------
IAP   :-------GVMFATTLTGEKASYVIQHCLEAWSAWGKPRIKTDNGPAYTSQKFRQFCRQMDVT-------
RSV   :-------IVVTQHGRVTSVAVQHHWATAIAVLGRPK-AIKTDNGSCFTSKSTREWLARWGIAH------
HTLV-1:-------SGAISATQKRKETSSEAISSLLQAIAHLGKPSYINTDNGRAYISQDFLNMCTSLA-------
HTLV-2:-------DTFSGAVSVSCKKKETSCETISAVLQAISLLGKPLHINTDNGPAFLSQEFQEFCT-------
BLV   :-------HASAKRGLTTQTTIEGLLEAIVHLGRPKKLNTDQGANYTSKTFVRFCQQFGVSLS-------
(energy = 866)
```

Fig. 2. An example move. This figure shows a move in our implementation of the simulated annealing algorithm: (a) initial solution and (b) first modified solution. In the initial solution, a number of gaps are added to both the head and tail of each sequence. The first move of a gap, for instance, modifies solution (a) to (b). Since the move reduces the energy of the solution, it is accepted. The energy is calculated from the similarity values in PAM250 with a linear gap penalty: $4 + n$, where n is the length of a gapped region.

that the temperature parallel algorithm gave better results on a graph-partitioning problem than the simple parallel algorithm (Kimura and Taki, 1990).

Implementation

We formulate the multiple sequence alignment problem, in order to apply the simulated annealing algorithm to it, as a combinatorial optimization problem, which requires the definition of a move to modify the current solution and an energy function to evaluate the solution.

Definition of moves

In our formulation, a basic move is defined as follows: (i) select one sequence in the alignment; (ii) select a gap and a column (horizontal location) randomly from that sequence; and (iii) move the gap to that column, to produce a candidate solution. For example, we start annealing from the initial solution in Figure 2(a). If the sequence RSV is chosen and the gap (represented by '−') at column 63 and column 37 (in this case, 'A') are selected, the gap moves to column 37 to give the solution in Figure 2(b).

In practice, we add a sufficient number of gaps to both sides of the alignment as shown in Figure 2. In a sense there are gap sources and sinks at both ends of each sequence. We refer to these special gaps as outgaps. For the sake of efficiency, we do not select any outgaps for the move except the ones immediately adjacent to letters (amino acids).

We have not only employed a move that depends on a single gap but also a move that depends on a cluster of gaps, which we refer to as a block operation. Since a good multiple alignment often contains clusters of gaps, the block operation accelerates the convergence of the annealing process. The operation has three modes: the vertical cluster mode, the horizontal cluster mode and the block cluster mode.

Let us apply the operations successively to the solution in

Fig. 3. Modification by block operations. A move of a vertical cluster of gaps modifies (a) to (b), a move of a horizontal cluster modifes (b) to (c), and a move to a block of gaps modifies (c) to (d).

Figure 3(a). In vertical mode, gaps in the same column move together to another selected column. When the number of gaps is equal to the number of sequences, the gaps are forced to the head or tail of the sequences and become outgaps. If a gap in column 21 and column 15 are selected in Figure 3(a), all five gaps in column 21 move to column 15, resulting in the solution shown in Figure 3(b). In horizontal mode, a row of gaps in a sequence move to another selected column in the sequence. If a gap in columm 38 and column 21 are selected in Figure 3(b), a row of three gaps at column 38 move to column 21, resulting in the solution shown in Figure 3(c). In block mode, rows of gaps at the same column move together to another place selected by the column number. If a gap in column 57 and column 63 are selected in Figure 3(c), the six rows of two gaps at column 57 move to column 63, resulting in the solution shown in Figure 3(d). In practice, we introduce moves with and without different types of block operations with equal probability.

Definition of energy function

Here, we only consider the alignment of protein sequences and define an energy function. Although the simulated annealing algorithm allows us to define a complex measure of similarity (see Discussion), we use one of the simplest measures here, as follows. The energy function is the sum of alignment scores for all pairs of sequences in the multiple alignment. Each alignment score is derived by summing up the similarity value at each position in two aligned sequences. The similarity value is given by the odds matrix of PAM250 (Dayhoff *et al.*, 1978).

Fig. 4. The optimal three-sequence alignment obtained by three-way dynamic programming. This alignment is similar to the one presented by Murata *et al.* (1985). The energy is calculated from the similarity values in PAM250 with a linear gap penalty: $8 + n$, where n is the length of a gapped region. The temperature parallel simulated annealing algorithm reproduced this optimal alignment.

We have reversed the sign of each value of the matrix so that the negative value is a gain and the positive value is a loss as in a regular energy function. The values range from -17 (W versus W) to 8 (W versus C).

We use a gap penalty dependent on the length of continuous gaps when calculating an alignment score for each pair of sequences. Specifically, we use a linear relation (Gotoh, 1982; Fitch and Smith, 1983) in the form of: $a + bn$, where n is the length of a gapped region and a and b are parameters. We set $a = 4$ and $b = 1$ for the problem shown in Figures 2, 3 and 7, and $a = 8$ and $b = 1$ for the problem shown in Figure 4. We assign the neutral value of zero to each position of the two aligned sequences when a gap is aligned to another gap or when a letter (amino acid) is aligned to an outgap.

Results

We report the results of two experiments. In the first experiment we confirm that the simulated annealing algorithm does provide the same optimal alignment as that obtained by the standard dynamic programming algorithm. In the second experiment we compare various methods and discuss advantages and disadvantages.

Experiment 1

We compared the temperature parallel simulated annealing algorithm with the rigorous three-way dynamic programming algorithm (Murata *et al.*, 1985; Gotoh, 1986; Hirosawa *et al.*, 1992). Figure 4 shows an optimal alignment for three sequences examined by Murata *et al.* (1985). They aligned the three sequences with a constant gap penalty, but we aligned them with a linear gap penalty as described above: $8 + n$, where n is the length of a gapped region. Therefore, there are some differences from the alignment reported by Murata *et al.* It took 74 min to align the sequences with three-way dynamic programming using our implementation on Multi-PSI (Hirosawa *et al.*, 1992).

The initial solution of the three sequences with no internal gaps was annealed by the temperature parallel algorithm with the same temperature assignment as described below. Figure 5 shows the histories of energy values obtained by the annealing

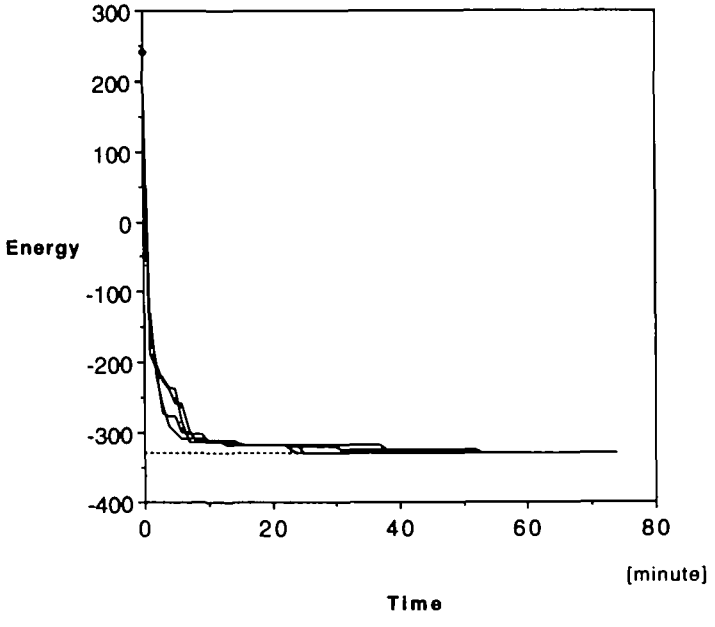

Fig. 5. The energy histories when aligning the three sequences shown in Figure 4 with the temperature parallel simulated annealing algorithm. Different histories were obtained with different random numbers. The horizontal dotted line indicates the energy level of the optimal alignment obtained by rigorous dynamic programming.

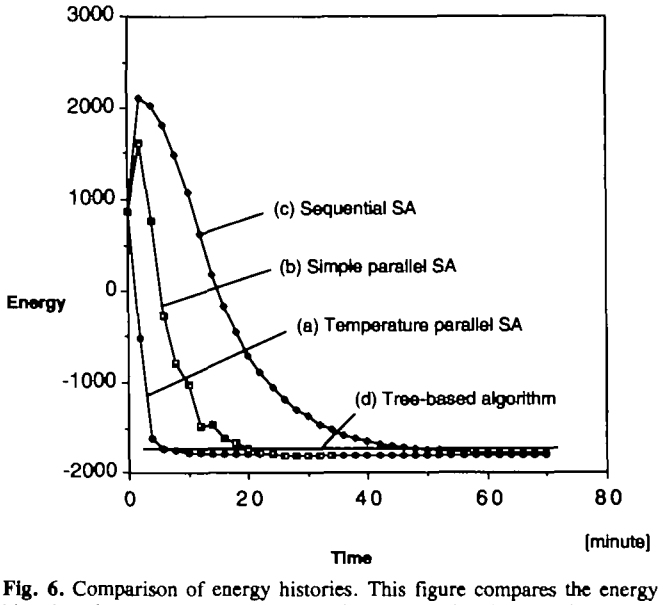

Fig. 6. Comparison of energy histories. This figure compares the energy histories when solving the problem in Figure 2 (a) with different algorithms. (a) In temperature parallel SA, each of the 63 PEs performed 20 000 annealing steps at a distinct constant temperature and the energy history at the lowest temperature is displayed. (b) In simple parallel SA, each of the 63 PEs executed sequential annealing of 10 000 steps and the best energy history of these is displayed. (c) Sequential SA shows the history of average energy for 20 000 annealing steps using a single PE. (d) The tree-based algorithm shows the energy level of the solution obtained by a conventional method.

algorithm with five different sequences of random numbers. The optimal solution obtained by the dynamic programming is indicated by the horizontal dotted line. The history of each annealing curve proves that the optimal solution is always obtained with the temperature parallel simulated annealing algorithm. It took 55 min on average to reach the optimal solution.

Experiment 2

Next, the initial solution of Figure 2(a) was annealed by the three simulated annealing algorithms: sequential, simple parallel and temperature parallel. We also experimented with a conventional tree-based algorithm. In Figure 6, the histories of energy values obtained by the annealing algorithms are shown, together with the solution obtained by the tree-based algorithm, represented by the horizontal line.

In temperature parallel SA, each of the 63 PEs performs 20 000 annealing steps at a distinct constant temperature. The highest and lowest temperatures are determined empirically from the equilibrium profile of the energy function. We used 100 and 0.3 in this case. The other temperatures are determined so that adjacent ones have roughly the same ratio, 0.9 or 0.8 in this case. Each point in Figure 6(a) represents the average of two runs with different sequences of random numbers.

In simple parallel SA, each of the 63 PEs executes 10 000 step sequential annealing using a distinct sequence of random numbers. The cooling schedule consists of exactly the same sequence of 63 temperatures as above. The history of energy for the best solution of those held by PEs is indicated in Figure 6(b).

For the sequential SA trial, 20 000 annealing steps are performed on a single PE with the same type of cooling schedule as above. Each point in Figure 6(c) represents an average of 63 runs with different sequences of random numbers.

The tree-based algorithm is a conventional method to produce rapidly a practical multiple alignment (Barton and Sternberg, 1987; Barton 1990; Higgins *et al.*, 1992). The algorithm aligns sequences one after another according to a tree-like order. Our implementation of the tree-based algorithm is as follows. First, a tree-like order is determined from a matrix of alignment scores calculated by dynamic programming for all possible pairs of sequences. Then, two-way dynamic programming is used to merge alignments following the tree-like order. When comparing two alignments, the score between two positions is calculated as the sum of similarity values for all character pairs taken from individual sequences in respective alignments. The definition of the similarity value is the same as described above. The tree-based algorithm is fast, but often provides a result with an energy value worse than the global minimum, as in this case shown in Figure 6(d).

The results of experiment 2 are summarized as follows.

(i) Both parallel algorithms gave the best solution shown in Figure 7(a) within 30 min. This alignment might correspond to a global minimum because annealing for another 30 min with temperature parallel SA did not improve it further.

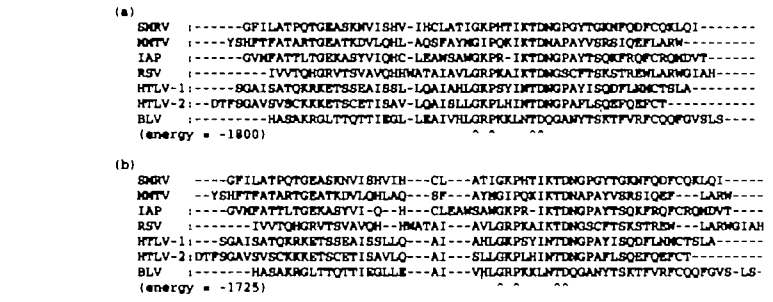

Fig. 7. The solutions for the problem shown in Figure 2(a). (a) The best energy solution generated by the parallel simulated annealing algorithms within 30 min. It might represent the optimal alignment. (b) The solution obtained by the conventional tree-based algorithm, which is worse than (a) in terms of the energy value.

(ii) The conventional tree-based algorithm solved the problem in 3 min, but the result shown in Figure 7(b) is worse than the result obtained by simulated annealing shown in Figure 7(a).

(iii) The sequential algorithm did not always find the optimal solution even within an hour, because an annealed solution may be trapped in a local minimum. On average, however, the final energy obtained was better than that by the tree-based algorithm.

Discussion

We have shown that the simulated annealing algorithm works well for simultaneous alignment of multiple sequences. The major disadvantage of long execution time can be alleviated by the use of a parallel computer. However, the alignment problem dealt with in experiment 2 has only 385 residues in total (55 residues for each of seven sequences), whereas a typical practical problem can have thousands of residues, 10 times as many as in our problem. The number of gaps in the solution of such a practical problem is supposed to be also 10 times as many. We can then roughly estimate that a practical problem would require a 100 (10 × 10) times more annealing steps to solve than the problem in this paper.

When we do not have enough time to execute complete annealing steps for a practical alignment problem, it is useful to regard the simulated annealing algorithm as a refinement tool. After we make an initial alignment using a heuristic method like the tree-based algorithm, we can often anneal up to the final alignment in a reasonable time. We could in fact reach the best solution shown in Figure 7(a) in 14 min using the parallel annealing algorithms starting from the approximate solution shown in Figure 7(b), which had been obtained by the tree-based method. In this case, the temperature parallel algorithm is also advantageous, because it works without designing a special cooling schedule for a partially aligned solution.

From a practical point of view, a major advantage of our methodology seems its flexibility to define a measure of similarity. It is possible to incorporate a context-dependent measure of similarity; for example, we can impose a higher gap penalty in a less homologous part of multiple alignment. It is also possible to incorporate additional contraints often utilized by human experts; for example, we can examine if patterns matching a specific sequence motif are well aligned. If the algorithm can be shown to produce mathematically rigorous and biologically meaningful alignments, the long execution time will then become a minor problem.

Acknowledgements

We thank Kouichi Kimura, the founder of temperature parallel SA, for valuable discussions. This work was partly supported by a Grant-in-Aid of Scientific Research on Priority Areas, 'Genome Informatics', from the Ministry of Education, Science and Culture of Japan.

References

Bacon,D.J. and Anderson,W.F. (1986) Multiple sequence alignment. *J. Mol. Biol.*, **191**, 153–161.

Barton,J.G. (1990) Protein multiple sequence alignment and flexible pattern matching. *Methods Enzymol.*, **183**, 403–428.

Barton,J.G. and Sternberg,M.J.E. (1987) A strategy for the rapid multiple alignment of protein sequences. *J. Mol. Biol.*, **198**, 327–337.

Carrillo,H. and Lipman,D. (1988) The multiple sequence alignment problems in biology. *SIAM J. Appl. Math.*, **48**, 1073–1082.

Dayhoff,M.O., Schwartz,R.M. and Orcutt,B.C. (1978) A model of evolutionary change in proteins. In Dayhoff,M.O. (ed.), *Atlas of Protein Sequence and Structure*. National Biomedical Research Foundation, Washington, DC, Vol.5, Suppl. 3, pp. 345–352.

Feng,D.-F. and Doolittle,R.F. (1987) Progressive sequence alignment as a prerequisite to correct phylogenetic trees. *J. Mol. Evol.*, **25**, 351–360.

Fitch,W.M. and Smith,T.F. (1983) Optimal sequence alignment. *Proc. Natl. Acad. Sci. USA*, **80**, 1382–1386.

Goad,W.B. and Kanehisa,M.I. (1982) Pattern recognition in nucleic acid sequences. I. A general method for finding local homologies and symmetries. *Nucleic Acids Res.*, **10**, 247–263.

Gotoh,O. (1982) An improved algorithm for matching biological sequences. *J. Mol. Biol.*, **162**, 705–708.

Gotoh,O. (1986) Alignment of three biological sequences with an efficient traceback procedure. *J. Theor. Biol.*, **121**, 327–337.

Higgins,D.G., Bleasby,A.J. and Fuchs,R. (1992) CLUSTAL V: improved software for multiple sequence alignment. *Comput. Applic. Biosci.*, **8**, 189–191.

Hirosawa,M., Hoshida,M., Ishikawa,M. and Toya,T. (1992) MASCOT: Multiple alignment system for protein sequences based on 3-way dynamic programming. *Comput. Applic. Biosci.*, **9**, 161–167.

Hogeweg,P. and Hesper,B. (1984) The alignment of sets of sequences and the construction of phyletic trees: An integrated method. *J. Mol. Evol.*, **20**, 175–186.

Johnson,M.S. and Doolittle,R.F. (1986) A method for the simultaneous alignment of three or more amino acid sequences. *J. Mol. Evol.*, **23**, 267–278.

Kimura,K. and Taki,K. (1990) Time-homogeneous parallel annealing algorithm. *Proc. Comp. Appl. Math. (IMACS '91)*, **13**, 827–828.

Kirkpatrick,S., Gelatt,C.D. and Vecci,M.P. (1983) Optimization by simulated annealing. *Science*, **220**, 671–680.

Murata,M., Richardson,J.S. and Sussman,J.L. (1985) Simultaneous comparison of three protein sequences. *Proc. Natl. Acad. Sci. USA*, **82**, 3073–3077.

Nakajima,K., Inamura,Y., Ichiyoshi,N., Rokusawa,K. and Chikayama,T. (1989) Distributed implementation of KL1 on the Multi-PSI/V2. *Proc. 6th Int. Conf. on Logic Programming*.

Needleman,S.B. and Wunsch,C.D. (1970) A general method applicable to the search for similarities in the amino acid sequences of two proteins. *J. Mol. Biol.*, **48**, 443–453.

Smith,T.F. and Waterman,M.F. (1981) Identification of common molecular subsequences. *J. Mol. Biol.*, **147**, 195–197.

Taylor,W.R. (1987) Multiple sequence alignment by a pairwise algorithm. *Comput. Applic. Biosci.*, **3**, 81–87.
Uchida,S. (1992) Summary of the parallel inference machine and its basic software. *Proc. Int. Conf. on Fifth Generation Computer Systems*, pp. 33–49.
Waterman,M.F., Smith,T.F. and Beyer,W.A. (1976) Some biological sequence metrics. *Adv. Math.*, **20**, 367–387.

Received on February 15, 1992; accepted on September 8, 1992